WalkMe React Native SDK — Integration Guide

`@walkme/react-native-sdk` is the React Native bridge for the WalkMe and WalkMe Power Mode (WalkMeEditor) SDKs on **iOS** and **Android**. This guide covers installing and integrating the bridge into a host app on both platforms.

---

1. Overview

- One JavaScript API (`WalkMeSDK`) bridges to the native SDK on both platforms.
- Two **flavors**: standard **WalkMe** and Power Mode **WalkMeEditor**. You pick the flavor per platform with a single declarative setting — no code changes.
- The bridge pulls the correct native SDK automatically (JitPack on Android, Swift Package Manager on iOS) and, on iOS, supplies the required Lottie dependency.

	Android	**iOS**
Min OS	Android 7.0 (API 24)	iOS 14
Native SDK source	JitPack (`com.github.WalkMe-int:…`)	Swift Package Manager
Flavor selector	`missingDimensionStrategy 'walkmeMode', …`	`"walkme": { "walkmeMode": … }` in `package.json`
Required RN version	any supported	**≥ 0.75** (for `spm_dependency`)

---

2. Install the bridge

```shell
npm install @walkme/react-native-sdk
# or: yarn add @walkme/react-native-sdk
```

The bridge is **autolinked** on both platforms — no manual native registration is needed.

---

3. Android setup

3.1 Add JitPack to your root `android/build.gradle`

```
None
allprojects {
    repositories {
        maven { url 'https://jitpack.io' }
    }
}
```

3.2 Choose a flavor in `android/app/build.gradle`

Add `missingDimensionStrategy` inside `defaultConfig`:

```
None
android {
    defaultConfig {
        // 'WalkMe' for the standard SDK, or 'WalkMeEditor' for Power Mode
        missingDimensionStrategy 'walkmeMode', 'WalkMe'
    }
}
```

The bridge automatically includes the matching WalkMe SDK — no extra `implementation` line is required. (It also bundles the Jetpack Compose dependencies the SDK needs.)

3.3 (Optional) Pin a specific SDK version

In your root `android/build.gradle`:

```
None
ext {
    walkmeVersion       = '1.2.3'  // used by the WalkMe flavor
    walkmeEditorVersion = '1.2.3'  // used by the WalkMeEditor flavor
}
```

If omitted, the latest published version is used.

---

4. iOS setup

> **Requires React Native ≥ 0.75.** The WalkMe iOS SDK ships only via Swift Package Manager, and the bridge pulls it in using RN's `spm_dependency` helper (added in RN 0.75). `spm_dependency` requires **dynamic frameworks**.

The bridge does the heavy lifting: it pulls the correct WalkMe SPM package **and** the matching Lottie dependency it needs, and ships the required CocoaPods `post_install` logic as a helper you call from your Podfile. You do **not** install `lottie-react-native`, set any environment variable, or copy any embedding script.

4.1 Select the flavor in `package.json`

Add a `walkme` block to your **app's** `package.json` (sibling of `dependencies`):

```JSON
{
  "dependencies": {
    "@walkme/react-native-sdk": "..."
  },
  "walkme": {
    "walkmeMode": "WalkMeEditor"
  }
}
```

`walkmeMode` value	**SDK pulled**
omitted, or `"WalkMe"`	standard **WalkMe** (default)
`"WalkMeEditor"`	Power Mode (**WalkMeEditor**)

The value is read at `pod install` time, is **case-insensitive**, and an unrecognized value (e.g. a typo like `"Editor"`) **fails the install with a clear error** — so you never silently build the wrong SDK. This mirrors the Android `walkmeMode` flavor: declare it once, in source control.

4.2 Wire up the `ios/Podfile`

Three additions, alongside what RN's template already generates:

```None
# (a) At the top, next to the react-native require — load the bridge's CocoaPods
```

```
#     helpers. Resolved via node so it's correct regardless of node_modules layout
#     (hoisting / Yarn or npm workspaces / pnpm), the same way RN resolves its
own.
require Pod::Executable.execute_command('node', ['-p',
  'require.resolve(
    "@walkme/react-native-sdk/scripts/walkme_podfile.rb",
    {paths: [process.argv[1]]},
  )', __dir__]).strip

# (b) The bridge requires dynamic frameworks (spm_dependency mandates this).
use_frameworks! :linkage => :dynamic

target 'YourApp' do
  config = use_native_modules!
  use_react_native!(:path => config[:reactNativePath])

  post_install do |installer|
    react_native_post_install(installer, config[:reactNativePath],
:mac_catalyst_enabled => false)

    # (c) Apply WalkMe's required build fixes: Lottie ABI + WalkMe SPM framework
embed.
    walkme_post_install(installer)
  end
end
```

> No `AppDelegate` changes are needed — `RCT_EXTERN_MODULE` auto-registers the module.

4.3 Install pods & run

```shell
npm install
cd ios && pod install && cd ..
npx react-native run-ios
```

A plain `pod install` selects the flavor from `package.json` and pulls Lottie via the bridge. To switch flavors later, edit `walkme.walkmeMode` and re-run `pod install`.

> **CI / one-off override:** the `WALKME_FLAVOR` environment variable still works and takes precedence over `package.json`, e.g. `WALKME_FLAVOR=WalkMeEditor pod install`.

---

5. Flavor summary

Flavor	Android (`android/app/build.gradle`)	iOS (`package.json`)	Native SDK
Standard	`missingDimensionStrategy 'walkmeMode', 'WalkMe'`	`"walkme": { "walkmeMode": "WalkMe" }` (or omit)	WalkMe
Power Mode	`missingDimensionStrategy 'walkmeMode', 'WalkMeEditor'`	`"walkme": { "walkmeMode": "WalkMeEditor" }`	WalkMeEditor

The native module name (`RNWalkMeSdk`) and the JavaScript API are identical regardless of flavor or platform.

---

6. Quick start (JavaScript)

```javascript
import WalkMeSDK from '@walkme/react-native-sdk';

WalkMeSDK.start({
  systemGuid: 'YOUR_SYSTEM_GUID',
  environment: 'Production',   // optional
  dataCenter: 'prod',          // optional: 'prod' | 'eu' | 'us01' | 'eu01' |
custom
});

WalkMeSDK.setUserId('user-123');
WalkMeSDK.setVariable('plan', 'premium');
WalkMeSDK.setLanguage('en');
WalkMeSDK.sendEvent('button_clicked', { screen: 'home' });
WalkMeSDK.stop();
```

See the package README for the full API reference.

---

7. How the iOS integration scripts work

The bridge ships **`scripts/walkme_podfile.rb`** inside the npm package and exposes one public function — `walkme_post_install(installer)` — that you call from your Podfile's `post_install`. It performs two fixes that **CocoaPods cannot do from a podspec alone** (a podspec can only configure its *own* pod target, not another pod or the app bundle). Keeping the logic in the bridge means it's version-locked to the SDK and never copy/pasted.

`walkme_fix_lottie_abi(installer)` — Lottie ABI

Sets `BUILD_LIBRARY_FOR_DISTRIBUTION = YES` on the `lottie-ios` pod. The prebuilt WalkMe frameworks are compiled against a **library-evolution (resilient)** build of Lottie, so they link Lottie's resilient symbols (e.g. `LottieLoopMode.loop`). The `lottie-ios` pod builds from source *without* library evolution, so those symbols would be missing and the app crashes at launch with `dyld: Symbol not found: …LottieLoopMode.loop` — even though `Lottie.framework` is embedded. Building Lottie with library evolution produces the matching ABI.

`walkme_embed_spm_frameworks(installer)` — embed the WalkMe framework

Rsyncs and codesigns the WalkMe SPM framework into the app bundle. `spm_dependency` links the framework to the Pods target but never embeds it in the app. On a **physical device** dyld only searches the app bundle, so without this the app aborts at launch with `dyld: Library not loaded: @rpath/WalkMeEditor.framework`. **Required for device/release builds.** (The simulator can load the framework from the build folder, so it happens to run without embedding — a device cannot.) The build phase is found-or-created by name, so re-running `pod install` never duplicates it.

Why Lottie comes from the bridge

The podspec declares `lottie-ios`, **pinned to the exact version the WalkMe frameworks were built against**, so a single standalone Lottie pod is shared. If your app *also* uses Lottie (e.g. via `lottie-react-native`), pin it to that same version so CocoaPods resolves one `Lottie.framework`; mismatched versions error rather than ship two copies.

> The only thing CocoaPods won't let the bridge do automatically is inject the `post_install` call itself (that would require a CocoaPods plugin). Hence the single `walkme_post_install(installer)` line in your Podfile.
